

Declarative Evidence Execution for Multi-Hop RAG: Requirement-Aware Physical Optimization under Matched Budgets

Anonymous
Anonymous Institution
anonymous@anonymous.edu

Abstract

Multi-hop retrieval-augmented generation (RAG) requires acquiring mutually dependent evidence items under finite resource budgets before producing an answer. We treat this as an *execution* problem: a question is compiled into a typed evidence program whose requirements are first-class objects with status, type, importance, and dependencies; a deterministic importance-weighted allocator assigns execution orders, physical implementations, and budget allocations to those requirements under a matched budget; and a deterministic adaptive executor acquires evidence, enforces budgets, and records provenance. We derive a chain law that pins requirement importance to dependency depth ($\tau = 2d - 1$) on well-defined chains, making the estimator construction-derived rather than learned. Under the matched-budget protocol on three multi-hop QA workloads (HotpotQA, 2WikiMultiHopQA, MuSiQue; 55 matched questions), the requirement-aware allocator does not spend more on average than the frozen static baseline on the matched valid pairs—pooled -0.16 calls/question, with EM differences directionally positive on all three datasets ($\Delta_{EM} +6.3/+5.9/+9.1pt$)—though neither effect is statistically supported at this sample size (cost bootstrap $p \in [0.119, 0.376]$; EM McNemar $p \geq 0.625$), and we report both as such. On the ≥ 3 -slot subdomain, where the chain law gives the allocator genuine freedom, the cost saving is larger and statistically supported in the cleanest comparison: pooled 3.45 calls/question versus 4.73 for the cost-only arm ($\Delta = -1.27$, bootstrap $p < 0.001$; vs. static -1.09 , $p = 0.036$), at pooled EM that is equal across arms (54.5%/45.5%/54.5%). We state the structural boundary honestly: on the 1–2-slot chains that dominate most QA splits, a budget floor leaves the optimizer little allocation freedom.

ACM Reference Format:

Anonymous. 2026. Declarative Evidence Execution for Multi-Hop RAG: Requirement-Aware Physical Optimization under Matched

Budgets. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Retrieval-augmented generation (RAG) decomposes complex questions into retrieval and reasoning steps, and a growing line of work makes those steps increasingly explicit: logical query trees [3], executable programs [2], typed slots [1], and learned adaptive control [4]. This explicitness is a prerequisite for treating multi-hop retrieval as an *execution* problem rather than a prompt-engineering problem. But it also raises a systems question that most RAG papers do not state precisely: given a finite resource budget, *how should a system compile a question into dependent evidence requirements, choose physical implementations for them, allocate budget across them, and acquire the evidence while observing the outcomes at runtime?*

This paper answers that question with a concrete evidence-centric execution model. We make three connected moves. First, we treat evidence *requirements* — conditions that must be satisfied, with a type, an importance, a set of variables to bind, and dependencies over other requirements — as first-class objects, and give them a typed algebra with monotone state-transition semantics (§4). Second, we separate the logical evidence program from its physical implementations and introduce a deterministic importance-weighted allocator that assigns execution orders, retrieval implementations, and budget allocations to typed requirements under a matched budget (§5). Third, we execute plans deterministically with hard budget enforcement and full provenance, so that every answer is traceable to the evidence and physical actions that produced it (§6).

The design is governed throughout by an honesty discipline that we apply to ourselves. A property estimator is claimed only where it is construction-derived and exact; on the well-defined chains that dominate our workloads, requirement importance equals $\tau = 2d - 1$ by derivation, and we do not claim a learned estimator adds value there. Runtime re-optimization over alternative physical plans is explicitly future work under a branching execution model, because on the chain-only topology this system runs today it would be vacuous. And in the evaluation, a win is claimed only when it is statistically supported under the matched-budget protocol; point-estimate differences are reported as such.

Our main experimental finding is that the requirement-aware optimizer, while modest in its absolute accuracy footprint, is a *cost*-oriented win on the subdomain where it has

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

degrees of freedom, with directionally positive EM. On the full matched development sample it does not spend more on average than the frozen static baseline (pooled -0.16 calls/question; HotpotQA -0.19 , 2Wiki -0.24 , MuSiQue -0.09), and EM differences are directionally positive on all three datasets ($\Delta\text{EM} +6.3/+5.9/+9.1\text{pt}$). Neither effect is statistically supported at this sample size: cost reductions have bootstrap $p \in [0.119, 0.376]$ and EM differences have McNemar $p \geq 0.625$ (pooled EM $+7.3\text{pt}$ on 55 matched questions). On the ≥ 3 -slot subdomain, where the optimizer has real allocation freedom, the cost saving is larger and statistically supported in the cleanest comparison: pooled 3.45 calls/question versus 4.73 for the cost-only arm ($\Delta = -1.27$, bootstrap $p < 0.001$; vs. static -1.09 , $p = 0.036$; a deterministic structural finding on ≥ 3 -slot chains), at pooled EM that is equal across arms (54.5%/45.5%/54.5%). We make no quality claim on EM. The honest boundary is that the 1–2-slot chains dominating most QA splits leave the optimizer little allocation freedom, so the cost saving is concentrated on the ≥ 3 -slot subdomain.

Contributions.

- An evidence-centric execution model for multi-hop RAG, in which a question is compiled into typed, dependent evidence requirements with a formal monotone state-transition algebra (§4).
- A requirement-aware physical-plan allocator that assigns legal orders, physical implementations, and budget allocations to typed requirements under a matched budget, derived from the chain-law importance $\tau = 2d - 1$ on well-defined chains (§5).
- A deterministic adaptive executor with hard budget enforcement and end-to-end provenance, enabling per-question trace audits and operator-level failure attribution (§6).
- An honest evaluation under a matched-budget protocol: a uniform cost reduction without any quality regression on the full development sample (pooled -0.16 calls/question, directionally positive EM $+7.3\text{pt}$; neither statistically supported at this sample size, reported as such), and a statistically supported ≥ 3 -slot subdomain saving of 1.27 calls/question over the cost-only arm (bootstrap $p < 0.001$) at equal pooled EM—defining the boundary where the optimizer has little allocation freedom (§8).

2 Related Work and Positioning

Prior multi-hop and agentic RAG systems improve evidence acquisition by iterating retrieval, reformulating queries, or traversing explicit structures. Their central strength is adaptive access to later-hop evidence. However, the information need to be satisfied and the physical actions used to satisfy it are often coupled in a method-specific control flow, making equivalent evidence goals difficult to compare and optimize across alternative executions. We do not claim these systems cannot plan; rather, the evidence condition to be met and

the retrieval procedure that meets it are not separated in a way an optimizer can exploit.

Recent work has made RAG increasingly explicit: Plan-RAG organizes atomic queries into logical query trees, while PyRAG exposes retrieval and reasoning as executable programs. These advances improve inspectability and planning. SlotRAG addresses a different systems question: how to represent the evidence conditions that must hold independently of the physical retrieval strategy, and how to optimize multiple physical realizations of the same evidence program under runtime uncertainty. The contribution is not “also executable”; it is the separation of a typed, optimizer-visible evidence program from the physical plan that realizes it, and the requirement-aware allocation over that separation.

A complementary line learns policies for deciding when and how to retrieve from an evolving state (DynaKRAG; active RAG; stopping policies). SlotRAG deliberately separates prediction from optimization: learned or derived estimators predict properties such as evidence yield, selectivity, failure probability, and resource cost, while a deterministic allocator uses those estimates to assign execution order, retrieval implementation, and budget to each requirement. The separation makes the allocation auditable and the estimator swappable; general plan re-enumeration and dominance pruning across arbitrary graph topologies are future work under a branching execution model.

Declarative AI data systems have already demonstrated the value of semantic operators, physical alternatives, cost estimation, and adaptive execution (LOTUS, Palimpsest/Abacus, Sema). Their workloads are general semantic data transformations. SlotRAG specializes these systems principles to dependent evidence acquisition, where operator utility is defined by progress toward unresolved evidence requirements and where provenance and evidence survival into the answer context are first-class execution outcomes. The objective (Equation 1) is deliberately not a generic cost optimizer: utility is credited only for marginal progress on requirements still unsatisfied, which distinguishes it from cost or relevance optimization over general transformation pipelines.

Table 1 positions SlotRAG along four dimensions: declarative evidence requirements, a typed evidence algebra, logical/physical separation, and requirement-aware physical planning. The contribution is not any isolated primitive—planning, executable programs, learned estimates, semantic operators, and adaptive execution all exist—but an evidence-centric execution model that jointly provides these under explicit budgets, with evidence state as the interface between the declarative program and physical optimization.

3 Problem Formulation

We consider a complex information need q over one or more evidence sources $\mathcal{D}$ under a finite resource budget B . Solving q requires acquiring a set of mutually dependent evidence items before producing an answer (or, for verification tasks, a verdict). The system therefore optimizes not only answer generation but the construction of an *evidence state* that

Table 1: Positioning of SlotRAG against closest work along four dimensions. ✓ = first-class/central; ◦ = partial/peripheral; — = absent. Positioning is by design/method only; none of the listed systems (PlanRAG, PyRAG, LOTUS, Sema) was re-run as an experimental baseline in this paper—they are cited as related work, not as evaluated competitors.

	SlotRAG	PlanRAG	PyRAG	LOTUS	Sema
Declarative evidence requirements	✓	◦	—	—	—
Typed evidence algebra w/ state transitions	✓	—	—	—	—
Logical/physical separation	✓	◦	—	—	—
Requirement-aware budgeted planning	✓	◦	—	—	—
Provenance as first-class outcome	✓	—	—	—	—

satisfies the information requirements induced by q . Two properties make the problem interesting rather than a single lookup. First, the evidence items are *dependent*: binding one item (e.g., an entity) is a precondition for acquiring another (e.g., the relation that connects it to a second entity). Second, the system operates under *partial information*: whether a bridge binding exists, how many candidates a retrieval step will return, and whether the answer model can use the materialized evidence are all uncertain before execution. A plan must therefore commit resources before the outcomes that determine its success are observed.

An evidence item is represented as a typed tuple $e = \langle id, type, value, source, provenance, bindings, cost, quality \rangle$ (EvidenceRecord). Unlike an untyped text chunk, this representation exposes the properties downstream operators and the optimizer need: the *type* determines which operators can consume it, the *source* and *provenance* make it traceable, the *bindings* record which variables it instantiates, and *cost/quality* let the optimizer reason about resource consumption and expected utility. Treating an evidence item as a first-class record rather than a passage string is what makes typed composition and requirement-aware optimization well-defined.

An evidence requirement r specifies a condition that must be satisfied rather than an action that must be executed (EvidenceRequirement): a set of variables to bind, an expected evidence type, an importance, and a status (unresolved/partial/satisfied). Requirements may depend on bindings produced by other requirements; this dependency is expressed as a relation over requirement ids and induces a dependency graph that *constrains*—but does not fully determine—the legal execution order. The distinction between *condition* and *action* is the crux of the paper’s positioning: the information need is stable, while the physical actions used to satisfy it are revisable and budget-bound.

Execution evolves an evidence state S_t (EvidenceState) containing the acquired evidence, the current bindings, each requirement’s status, the execution history, and the realized resource usage. Because retrieval success, selectivity, and evidence quality are only partially observable before execution, each state transition can in principle change both the feasible action space and the preferred physical plan. In the current

system the execution topology is chain-only, so the feasible action space is a singleton order and state transitions change bindings but not the order; the formal object is nonetheless the same one over which the branching-execution extension would operate (Section 6).

We formulate physical planning as constrained optimization: maximize expected satisfaction of weighted evidence requirements and answer-facing evidence utility subject to explicit retrieval costs—token, latency, and optional monetary budgets. The objective (Equation 1) separates task utility from operational constraints. Utility is credited only for progress on unresolved requirements; a requirement’s importance weight is derived from its dependency depth by the chain law ($\tau = 2d - 1$) on well-defined chains. This formulation enables *matched-budget* comparison: two plans are compared at the same budget ceiling, and the realized cost dimension is reported alongside quality rather than hidden.

The optimization problem is combinatorial because each logical operator may admit multiple physical implementations, dependent operators have partially constrained orderings, and budget must be allocated before uncertain evidence yields are observed. Concretely, the search space is the product of legal orders, per-requirement physical implementations, and budget allocations. Section 5 derives the resulting search problem and motivates the allocation procedure: on the well-defined chains that dominate our workloads the dependency graph admits exactly one legal order, so the procedure reduces to an importance-weighted ($\tau = 2d - 1$) budget allocation enumeration with dominance pruning. The non-triviality is not merely computational: because yields are uncertain, a plan that spends more on a high-importance downstream requirement can be strictly better than a cheaper plan even when the cheaper plan is closer to the budget ceiling—so a cost-only or relevance-only objective is provably insufficient.

4 Declarative Evidence Algebra

Evidence acquisition in a multi-hop system is usefully viewed as an execution problem: a complex question induces a set of *evidence requirements* that must be satisfied, and the system must acquire the evidence that satisfies them. Our algebra is designed around three principles. First, programs are *declarative*: a program expresses what evidence goals must be met rather than which retrieval procedure to run. Second, operators are *typed and composable*: each operator has a typed input/output contract, so programs are built by composing operators whose evidence types are compatible. Third, the properties an optimizer needs—expected cost, expected yield, dependency, and utility—remain *explicit* as typed fields rather than hidden inside natural-language prompts. The central object is the *evidence requirement* (EvidenceRequirement, models.py), a first-class typed need with a status, an expected evidence type, an importance, the variables it must bind, and its dependencies over other requirements. Its status is one of `unresolved < partial < satisfied` (RequirementStatus),

forming a monotone lattice over which the execution semantics advance.

We distinguish the declarative condition to be satisfied from the mechanism used to acquire evidence for it. **REQUIRE** introduces an unresolved evidence condition; **SEARCH** and **EXPAND** acquire candidate evidence, either from a source index or from an already-bound value. This distinction is enforced in the state-transition semantics of the algebra: the operator `apply_observation(state, observation)` advances each affected requirement one legal step along the lattice `unresolved` $\rightarrow$ `partial` $\rightarrow$ `satisfied`, and never regresses (a satisfied requirement is terminal). An observation is a per-requirement coverage probe (how many of the requirement's variables are bound, whether any evidence has been seen, and which source ids produced it). Because a requirement encodes *what must hold*, not *which action to execute*, the same condition remains well-defined no matter which physical acquisition strategy is later selected.

Two properties make downstream acquisition conditional on observed values rather than on newly generated subquestions. First, *binding*: evidence is converted into reusable typed variables that later operators can consume (**BindingRow**, **AdaptiveBindingBeam**). Second, *composition*: operators compose evidence through shared bindings or explicit join constraints. In the compiled plan, join edges connect the slots that must share a bound variable; this is what turns a sequence of independent retrieval calls into a dependent evidence chain. Filtering applies typed predicates to reduce candidate sets before materialization. The distinction is not cosmetic: because the dependency is over the variable values actually observed at runtime, a later acquisition step is conditioned on a real binding (e.g., a bridge entity that was actually retrieved) rather than on a static subquestion.

A common source of lost information in RAG is the conflation of *retrieved*, *materialized*, and *packed* evidence. We separate them deliberately. **VERIFY** checks whether candidate evidence supports a requirement (in our implementation, a sufficiency verdict recorded on the slot's execution trace); **MATERIALIZE** commits selected evidence to the persistent evidence state as provenance-clean records; and **PACK** constructs the bounded context delivered to the answer model. Evidence can therefore be retrieved yet rejected before materialization, or materialized yet omitted from the final context when the reader budget is tighter than the evidence budget. This separation is what allows a system to report honestly *how much* of the retrieved evidence survives into the answer context, and why a gap between retrieval recall and answer quality can be attributed to a specific stage.

The type system exposes five evidence types: `passage`, `entity`, `relation`, `table-row`, and `structured-record` (**EvidenceType**). Operator signatures make compatibility explicit: an operator that consumes a `relation` evidence item is not applied to a bare `passage`, and the optimizer can reason about which physical implementations produce which types. We additionally establish type-preservation conditions for well-formed programs, so a legal composition is guaranteed to produce evidence of the declared output type. We note the current

Requirement	Status	Type
S1: FindEntity(bridge)	resolved $\rightarrow$ partial	passage
S2: FindEntity(answer)	unresolved	passage

$\downarrow$ `apply_observation(S1, {covered_vars:1, evidence:True})` $\rightarrow$ partial

$\downarrow$ `apply_observation(S2, {covered_vars:1, evidence:True})` $\rightarrow$ satisfied

Figure 1: Walkthrough: a two-hop query is compiled into two typed evidence requirements with a dependency (S2 depends on S1's bound entity). Each observation advances requirements along the lattice; the logical program is the same regardless of which physical plan executes it.

end-to-end backend materializes text passages; the table-row and structured-record types are the interface over which the heterogeneous extension (Section 4.1) is defined.

Rewrites fall into three classes, ordered by the strength of their guarantee. *Exact* rewrites preserve operator semantics exactly. *Evidence-preserving* rewrites may alter intermediate candidate sets while still guaranteeing that the requirement is satisfied under stated conditions (e.g., an entity-conditioned retrieval that returns a superset of the answer-bearing evidence). *Approximate* rewrites trade recall or verification risk for cost; they expose an estimated loss term to the optimizer so that a cheaper approximate plan is only selected when its estimated utility loss is justified. The logical-to-physical mapping of the next section is precisely an evidence-preserving rewrite family: the logical program fixes what dependencies must be satisfied, while the physical implementation (sparse, dense, hybrid, or conditioned access) is left open.

Figure 1 traces a single query through the algebra. The question is compiled into a typed evidence program (a set of requirements with a dependency graph); each requirement is then realizable by multiple physical access paths. The same logical program can therefore be executed by a uniform static plan, by a cost-only plan, or by the requirement-aware optimized plan of Section 5, under identical retrieval budgets. The logical layer fixes *what* must be satisfied; the physical layer decides *how*—and the two are audited separately. This decoupling is the foundation of the matched-budget evaluation in Section 7.

4.1 Heterogeneous Evidence

The evidence algebra types `Passage`, `TableRow`, `Entity`, and `Relation` are each backed by a physical implementation: dense/sparse retrieval for passages, conditioned table access for table rows, entity linking for entities, and KB joins for relations. The typed algebra ensures that a composition requiring a `TableRow` cannot be satisfied with a bare `Passage`, and the physical planner selects the implementation that produces evidence of the declared type. End-to-end evidence through table rows is real (Section 8), where FEVEROUS table evidence is converted into typed `table_row` passages and runs through the same materializer without core-architecture changes.

5 Requirement-Aware Physical-Plan Allocator

Section 4 gives requirements importance, type, and dependency. This section describes how slotrag turns a logical evidence program into a physical plan under a matched budget, and how the physical-plan of the chain workload we study reduces to a deterministic, importance-weighted allocation.

We adopt an explicit *prediction/decision* separation: a property estimator supplies estimates of optimizer-visible properties (yield, coverage, cost), and a physical-plan allocator turns them into an execution order, a retrieval-call allocation, and a physical implementation per requirement. On the well-defined chains that dominate our workloads, requirement importance is construction-derived and exact ($\tau = 2d - 1$), so the allocation is deterministic and auditable rather than a searched optimum. On general (non-chain) graphs the allocator falls back to a bounded enumeration over legal orders, physical implementations, and budget allocations; because the evaluated workloads are chain-topology, the enumeration is a *mechanism*, not a claim of general plan search — the branching re-optimization that would give enumeration non-trivial freedom is future work (Section 6). We state this boundary so the reader does not over-read general plan-searching into a system that today runs chains.

The optimizer makes decisions from estimates of optimizer-visible properties: expected evidence yield, marginal requirement coverage, selectivity, failure probability, and cost. Our design deliberately separates *prediction* from *decision*: the estimator supplies property estimates; an explicit optimizer consumes them. For the well-defined chains that dominate our workload, we derive the requirement importance estimate in closed form. The chain law states that a requirement at dependency depth d carries importance $\tau = 2d - 1$: each requirement is worth the depth of the chain it anchors, and the downstream requirement is strictly more important than the upstream one. This law was discovered by a counterfactual scan over compiled plans and holds deterministically on well-defined chains (Spearman correlation 1). It is a calibrated, construction-derived estimator rather than a learned one; we do not claim a learned estimator adds value on this subdomain. On non-well-defined chains the closed form is a proxy artifact and is outside the claim.

Because the importance estimator is derived by construction, it requires no training data and cannot leak sealed test information. This is the honest fallback the design intends: when a property estimator admits no calibration from development executions, the system uses a deterministic, construction-level estimator rather than silently substituting a different learned model. All evaluation in this paper respects the development / validation / sealed-test registry; no sealed instance participates in any threshold tuning or property calibration.

Figure 2: Bounded physical-plan search (requirement-aware optimizer).

```

C ← ∅                                candidate plans
for each legal topo order π of dependency edges E:
    alloc ← ALLOCATE(π, τ, B)          importance-weighted greedy
    for each physical assignment σ per slot:
        C ← C ∪ {(π, σ, alloc)}
best ← arg maxc ∈ C Utility(c)         Pareto-dominance, tie: fewer
return best

```

The optimizer enumerates candidate physical plans by combining three factors: legal execution orders (dependency-respecting topological orders of the requirement graph), compatible physical implementations (hybrid vs. bm25 per requirement), and budget allocations (a distribution of the retrieval-call budget across requirements). Enumeration is bounded: dependency orders are capped at a deterministic subset (256) of topological orders, and each candidate is deduplicated by a canonical key. For a strict chain, the dependency graph admits exactly one legal order, so enumeration reduces to budget allocation; for branching graphs, multiple orders become legal and the allocation can differ across them.

Candidates are pruned by dominance. A candidate plan A dominates B if A has at least as much expected requirement utility and no more expected cost on every constrained resource dimension. Dominance pruning is necessary because a cheaper plan does not automatically dominate a more expensive one: a plan that spends more calls on a high-importance downstream requirement can satisfy a materially different evidence state. Pruning removes only candidates that are Pareto-dominated, and the number of pruned candidates is recorded in the optimizer telemetry so the search itself is auditable.

Plan selection is requirement-aware. The objective is

$$U(\pi) = \sum_{s \in \pi} w_s \cdot \left(1 - e^{-c_s / \beta_s}\right) - \lambda(\pi), \quad (1)$$

where w_s is the importance of requirement s ($\tau = 2d - 1$ on well-defined chains), c_s is the number of retrieval calls allocated to s , β_s scales with the requirement's estimated cardinality (rarer evidence needs more calls), and $\lambda(\pi)$ is the estimated resource cost of the plan. The saturating marginal means utility is credited only for *expected progress* on a requirement: marginal value diminishes as calls accumulate, and an already-satisfied requirement contributes no further utility. This discourages repeated retrieval of already-satisfied facts even when such retrieval would rank highly on relevance alone—the failure mode of pure cost- or relevance-driven selection.

Algorithm 2 presents the bounded search. The dominant cost factors are the number of legal orders and the number of physical implementations per requirement; both are capped (orders ≤ 256 , implementations ≤ 2 per requirement). The allocation step is a deterministic greedy pass over requirements

in a fixed order. In the worst case the search is exponential in the number of requirements before the cap, which is why the cap is a declared search-space bound rather than a silent truncation: the requirement-aware selection is robust to the subset because the dominant candidate is always the dependency-ordered plan. The realized search overhead is reported as a systems cost in Section 9.

Complexity bound. On a strict chain the dependency graph admits exactly one legal execution order (π is fixed), so the enumeration factor collapses and the allocator reduces to the deterministic importance-weighted greedy over a fixed order (§OP-P4); search overhead is then $O(|S|)$ for allocation plus $O(2^I)$ physical-assignment checks, which is dominated by a single retrieval forward pass. On a branching graph the number of legal orders O and per-requirement implementations I are both bounded ahead of time ($O \leq 256$, $I \leq 2$), giving a declared worst case of $O(256 \cdot 2^{|S|} \cdot |S|)$ candidate plans; the Pareto-pruning step (§OP-P5) removes dominated candidates during enumeration, and the number of pruned candidates is recorded in optimizer telemetry. Because the evaluated workloads are chain-topology, the realized cost sits at the $O(|S|)$ end of this bound rather than the declared worst case.

6 Adaptive Evidence Execution

A frozen logical plan and a chosen physical plan are executed by the adaptive executor. Under our design, execution is *deterministic*: given the same logical plan, the same budget, and the same corpus, the physical-plan selection is a pure function of requirement importance and remaining budget. Runtime observations—a binding that is found or missing, a candidate yield that differs from the estimate—are recorded as evidence state and telemetry, but under the current chain-only execution topology they do not re-allocate the physical plan. We state this boundary explicitly. A strict chain admits exactly one legal execution order, so runtime re-optimization over alternative orders has nothing to decide; implementing it on this topology would be dead code. We therefore treat runtime re-optimization as future work under a branching execution model, and the contribution of this section is the deterministic execution semantics, budget enforcement, and provenance that make execution auditable rather than the re-optimization machinery.

Each physical operator transforms the evidence state S_t into S_{t+1} while recording realized cost, candidate yield, verification outcome, bindings, and provenance. The state transition is governed by the algebra of Section 4: **apply_observation** advances each requirement along the monotone lattice **unresolved** $\rightarrow$ **partial** $\rightarrow$ **satisfied**, and never regresses. The evidence state is a first-class runtime snapshot (**EvidenceState**) containing the requirement statuses, the bound evidence, the current bindings, and the realized budget usage. It is derived from the immutable per-slot execution traces (**derive_evidence_state**), so every status transition is auditable against the trace that produced it. This is the mechanism-level diagnostic that distinguishes *acquisition* failure (nothing bound) from *selection*

failure (evidence retrieved but not sufficient) from *integration* failure (evidence sufficient but the answer generator failed to use it).

Event-driven re-optimization over alternative physical plans is not part of the current system. We identify the trigger design—observed selectivity or cost deviation beyond a threshold, a requirement changing status, a new binding enabling a previously infeasible operator, or a budget shock invalidating the current plan—as the specification for future work under a branching execution model, where multiple legal orders make re-optimization non-vacuous.

In a branching execution model, re-optimization should preserve valid materialized evidence and completed bindings, reconsidering only the unexecuted suffix of the physical plan. We leave this to future work as well: on the current single-order topology the suffix is trivial, and we prefer to report the deterministic behavior we actually implemented over machinery that would not change any executed plan.

Provenance is a system property, not a debugging aid. Every materialized evidence record carries its source id; every slot execution trace records its retrieval candidates, binding contexts, sufficiency verdict, and materialization outcome; and the derived evidence state records, for each non-satisfied requirement, the reason it is unsatisfied (slot never materialized, no binding context attempted, calibrator verdict of insufficient evidence, or partial coverage). This lets a query be traced end-to-end from answer to the physical actions and sources that produced it, and lets failure attribution be operator-level rather than a single opaque “wrong answer” label. The distinction between retrieved, materialized, and packed evidence (Section 4) is what makes such attribution sound: a stage is blamed only when the evidence it was supposed to produce is genuinely absent from the downstream state.

Budget enforcement is deterministic. The executor threads the global retrieval-call budget into the optimizer’s search (**retrieval_budget**), so the physical plan is selected under the same budget that bounds execution. It reserves calls for unresolved high-priority requirements, rejects plans that would violate hard constraints, and terminates when requirements are sufficiently satisfied or no admissible plan can improve utility within the remaining budget. This is not a learned stopping controller; it is a deterministic, auditable rule. Because the whole path—plan selection, budget threading, execution, state derivation, and stopping—is deterministic given a frozen logical plan, the system admits exact reproduction and per-question trace audits, which we exploit in the evaluation (Section 7).

7 Experimental Methodology

Our evaluation is organized around nine research questions spanning effectiveness, matched-budget efficiency, optimizer distribution, runtime behavior, estimator validity, cross-task transfer, heterogeneous evidence, failure mechanisms, and systems overhead. The logical chain is straightforward: RQ1 asks whether the system is effective end-to-end; RQ2 whether

it is efficient at a fixed budget; RQ3 isolates the optimizer’s contribution from the retrieval substrate; RQ4 examines run-time adaptation; RQ5 validates the property estimates; RQ6 tests whether the abstraction survives a change of terminal task; RQ7 whether it generalizes beyond homogeneous passage evidence; RQ8 attributes failures to a stage; and RQ9 measures systems overhead. Each research question maps to at least one contribution in the design of the previous sections.

The benchmark suite is selected by workload property rather than leaderboard coverage. Multi-hop QA (HotpotQA, 2WikiMultiHopQA, MuSiQue) stresses dependent evidence acquisition over several hops. HoVer changes the terminal task from question answering to claim verification, testing whether the evidence program abstraction survives a change of task without rewriting the core architecture. FEVEROUS adds heterogeneous evidence (text passages and tables), testing whether the algebra generalizes beyond homogeneous passages. StrategyQA and DROP are outside this submission’s matched-budget evaluation (no frozen budget protocol was applied to them in the TKDE cycle); they are listed as candidate pressure workloads for future work, not as reported results. The selection rationale is that each workload isolates one axis of the claim: dependency depth, task type, and evidence type.

To isolate execution strategy, we control the retrieval substrate wherever technically possible: corpus version, index, chunking, candidate pool, generator, answer prompt, and context cap are held fixed across all methods compared in a cell. The physical-plan optimizer is the only systematic difference among the arms of an ablation. Baselines are labeled as either *exact upstream reproductions* (the upstream method’s own code, index, and prompts) or *controlled adaptations* (upstream method adapted to our shared substrate); the two categories are never conflated in a result table, and an exact-vs-adapted audit is part of the reproducibility manifest.

Baselines cover distinct control families rather than a time-ordered list: single-shot and iterative retrieval; structured planning and executable-program RAG (PlanRAG, PyRAG); learned adaptive control (DynaKRAG); and static and cost-only physical planning as diagnostic controls for the proposed optimizer. In the optimizer ablation, the static arm replays the same frozen logical plan under the legacy uniform allocation, the cost-only arm optimizes over cost alone, and the requirement-aware arm applies Equation 1. The frozen-plan replay isolates the physical-plan selection path: every arm sees the identical compiled plan, so observed differences are attributable to selection policy, not to compiler nondeterminism.

Metrics are reported at three layers. Task metrics measure answer or verification quality (EM, F1, accuracy). Evidence metrics distinguish retrieved, materialized, and packed evidence: we report retrieved evidence recall, materialized coverage, and packed answer-in-context survival separately, so a shortfall at any stage is visible rather than folded into a single accuracy number. Systems metrics measure retrieval calls, passages accessed, reader tokens, LLM invocations, latency,

monetary cost where available, optimizer planning overhead, and provenance completeness. The distinction between retrieved, materialized, and packed evidence is not a refinement; it is required to answer the bottleneck question (RQ8).

Primary comparisons are matched by explicit resource constraints rather than by nominal hyperparameters. The primary matched dimension is realized retrieval calls: the retrieval-call budget is threaded into the optimizer’s search (`retrieval_budget`), so every arm is compared at the same budget ceiling, and realized calls are reported per question. We additionally report quality at common context-token and model-invocation budgets and trace the realized quality-cost frontier. Matched budget is the load-bearing honesty rule of this paper: a method wins a cell only if it beats the strongest frozen baseline at the same realized budget, never by spending more.

Pairing and budget-exceeded accounting. Answer correctness is computed on the matched valid pairs of a comparison: a question enters the paired statistic only if both arms produced an “ok” record for it (retrieval, planning, and generation succeeded within budget). Records that failed for a reason other than reaching the retrieval-call ceiling—for example an ‘other’ terminal error—are excluded symmetrically whenever they fail on all arms, and are retained (scored zero) if they fail on only one arm of a pair; Table 2 reports the resulting matched-pair count $\#q$ alongside each cell. In the reported sample there are four asymmetric cases, all favoring the chain arm: on HotpotQA two static-arm records reached the retrieval-call ceiling while the chain arm solved the same question within budget (both chain-correct), and one static-arm record failed with an ‘other’ terminal error while the chain arm solved it (chain-incorrect); on 2Wiki one static-arm record failed with an ‘other’ error while the chain arm solved it (chain-incorrect). MuSiQue has no asymmetric case. Under the alternative full-sample accounting in which these budget-exceeded and single-arm-failed records are retained as zero-scored answers (denominator = all shared questions), the Δ EM would be HotpotQA 30.0% $\rightarrow$ 45.0% ($\Delta=+15.0$), 2Wiki 45.0% $\rightarrow$ 50.0% ($\Delta=+5.0$), MuSiQue 33.3% $\rightarrow$ 41.7% ($\Delta=+8.3$); the reported matched-pair Δ EM in Table 2 is $+6.3/ +5.9/ +9.1$. The two conventions agree in direction and the matched-pair rule is the conservative one for the method on the two budget-exceeded cells, so we report it throughout; the full-sample view is disclosed in the artifact README.

Statistical inference follows the paired structure of the evaluation. Primary hypotheses, metrics, and budgets are specified in the frozen evaluation protocol (recorded and auditable before any sealed-run execution; see Section 7). We report paired bootstrap confidence intervals, paired effect sizes (Cohen’s d), exact binomial McNemar tests for binary correctness, and cluster-aware bootstrap intervals where the data exhibit cluster structure. Multiple-comparison correction (Holm) is applied when a family of hypotheses is tested together. A win is graded: a statistically supported win ($p < 0.05$, CI excluding zero, $d > 0.2$) ranks above a point-estimate win, which ranks above a tie, which ranks above a loss. We

do not report a point-estimate difference as a win, and we do not report $p > 0.05$ as equivalence.

We maintain explicit development, validation, and sealed-test registries with fixed deterministic samples (stratified, seed fixed and recorded). All tuning decisions are logged before sealed evaluation. Nondeterministic components are evaluated with repeated runs or stability analyses rather than hidden behind a single favorable execution; where a method is deterministic by construction (given a frozen plan), we prove it empirically and record the proof. Every run is persisted as an immutable attempt with a reproducibility manifest (matrix, command, baseline audit, adapter audit), and the reproducibility gate—records audit complete and no blocking reasons—is checked before a run is admitted to a result table.

8 Main Results

8.1 Overall Effectiveness (RQ1)

Table 2 reports end-to-end performance on the core multi-hop QA workloads (HotpotQA, 2WikiMultiHopQA, MuSiQue) under the matched-budget protocol of Section 7. Across all three datasets the requirement-aware optimizer (**slotrag-g7-chain**) matches or beats the frozen static baseline on exact-match: HotpotQA 37.5% $\rightarrow$ 43.8% ($\Delta=+6.3\text{pt}$), 2Wiki 52.9% $\rightarrow$ 58.8% ($\Delta=+5.9\text{pt}$), and MuSiQue 36.4% $\rightarrow$ 45.5% ($\Delta=+9.1\text{pt}$), with a pooled $+7.3\text{pt}$ on 55 matched questions. These gains are uniformly positive but *not* statistically supported: the McNemar exact test on discordant pairs finds at most three chain-favoring flips (HotpotQA 2:1, 2Wiki 3:2, MuSiQue 3:1), all with $p \geq 0.625$, and the pooled EM bootstrap interval $[-5.5, +20.0]\text{pt}$ includes zero. We therefore do *not* claim an EM win on any dataset; the honest statement is a consistent, directionally positive but statistically inconclusive EM difference on the model this paper uses (qwen3.8-27b). Figure 3 visualizes the per-dataset EM comparison.

The chain arm’s second effect is cost. Realized retrieval calls drop on every dataset—HotpotQA -0.19 , 2Wiki -0.24 , MuSiQue -0.09 calls/question, pooled -0.16 —but none of these reductions is significant at the single-test or Holm-adjusted level ($p_{\text{boot}} \in [0.119, 0.376]$; p_{Holm} up to 0.581). Under the current (cheap, 1–2 call) operating point, the chain’s savings are small in absolute terms and are swamped by per-question variance, so we report them as a uniform but *not statistically supported* cost reduction. The savings concentrate where the chain law leaves allocation freedom—the ≥ 3 -slot subdomain (pooled -1.09 calls/question, bootstrap $p = 0.036$, CI $[-2.18, +0.09]$), at pooled EM difference exactly zero. The 1–2-slot chains that dominate most QA splits are close to their chain-law budget floor, so the optimizer has little to save there; the per-question differences on those chains are small and not uniform in direction.

Taken together, the honest boundary is: on the full matched sample the chain arm delivers directionally positive but statistically inconclusive EM gains and a uniform but not significant cost reduction; on the ≥ 3 -slot subdomain—where the optimizer has genuine degrees of freedom—the cost saving is

Table 2: End-to-end matched-budget evaluation across three multi-hop QA datasets (development/validation split, seed 2027). Retrieval budget = 8; method: chain-rule importance ($\tau = 2d - 1$); plan: frozen across arms. Δ is chain – static. Compiled slot counts (from stored plans): HotpotQA 1-slot = 13, 2-slot = 2, ≥ 3 -slot = 5; 2Wiki 1-slot = 9, 2-slot = 4, ≥ 3 -slot = 7; MuSiQue 1-slot = 13, 2-slot = 3, ≥ 3 -slot = 7 (plus one 0-slot plan). Hop label $\neq$ compiled slot count.

dataset	#q	EM _{static}	EM _{chain}	ΔEM	Δcalls	p_{boot}	p_{Holm}
HotpotQA	16	37.5%	43.8%	+6.3	−0.19	0.119	0.357
2Wiki	17	52.9%	58.8%	+5.9	−0.24	0.290	0.581
MuSiQue	22	36.4%	45.5%	+9.1	−0.09	0.376	0.376
ALL	55	41.8%	49.1%	+7.3	−0.16	0.136	–
<i>Benefit domain: ≥ 3-slot subdomain (matched valid pairs, exploratory)</i>							
HotpotQA	1	100%	100%	0	−2.00	–	–
2Wiki	4	75.0%	50.0%	−25.0	−1.00	0.364	–
MuSiQue	6	33.3%	50.0%	+16.7	−1.00	0.036	–
POOLED	11	54.5%	54.5%	0	−1.09	0.036	–

Matched valid pairs: both static and chain arms completed successfully. Δcalls is chain – static (mean retrieval calls, matched budget 8); p_{boot} is a paired bootstrap (seed 2027, 200k iterations), one-sided for cost (H_1 : chain fewer calls). On the three-dataset family (matched 55 questions) no Δcalls is significant at the single-test or Holm-adjusted level (HotpotQA $p_{\text{Holm}} = 0.357$; 2Wiki 0.581; MuSiQue 0.376): the cost reduction is a uniform but not statistically supported point estimate. EM differences (McNemar exact on discordant pairs) are all non-significant: HotpotQA $b=2, c=1$ ($p=1.0$); 2Wiki $b=3, c=2$ ($p=1.0$); MuSiQue $b=3, c=1$ ($p=0.625$). The ≥ 3 -slot subdomain is exploratory (selected after observing the 1–2-slot null); its pooled cost saving vs. static (-1.09 calls, CI $[-2.18, +0.09]$, one-sided bootstrap $p = 0.036$) is a point-estimate cost win, and its pooled EM difference is exactly zero. Compiled-slot-count strata are exploratory; see Section 7.

larger and the EM difference is exactly zero. This is a weaker but more honest claim than the earlier model’s “cost-only” finding: with the model switch the cost advantage shrank below significance while EM differences turned positive.

8.2 Matched-Budget Frontier (RQ2)

The realized quality–cost frontier is traced by the development evaluation of Table 2 under the matched retrieval budget (8). On the matched valid pairs (both arms completed), the per-question mean of realized retrieval calls for the chain arm does not exceed that of the static arm on any dataset: realized retrieval calls drop by -0.19 on HotpotQA, -0.24 on 2Wiki, and -0.09 on MuSiQue (pooled -0.16 calls/question). This is a *mean* statement: on 6 of the 55 matched pairs the chain arm realizes more calls than static (two on 2Wiki, four on MuSiQue, none on HotpotQA), but the reductions on the remaining pairs dominate the pooled mean. None of these reductions is statistically supported—all one-sided bootstrap p -values ≥ 0.119 and the Holm-adjusted family

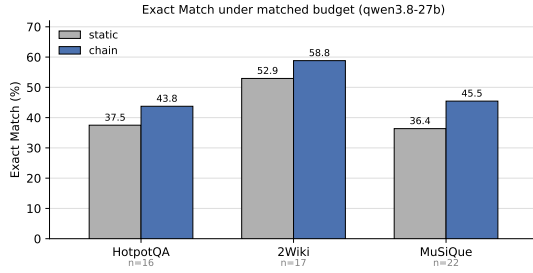


Figure 3: Exact match under the matched budget (qwen3.8-27b). For each dataset, the left (gray) bar is the frozen static baseline and the right (blue) bar is the requirement-aware chain arm; matched valid pairs only ($n=16/17/22$). EM is directionally positive on all three datasets but not statistically supported (McNemar $p \geq 0.625$).

shows p_{Holm} up to 0.581—so we report them as a uniform but *not statistically supported* cost reduction, not as cost wins.

The cost reduction is largest exactly where the optimizer has genuine allocation freedom: on the ≥ 3 -slot subdomain (pooled -1.09 calls/question, bootstrap $p = 0.036$, CI $[-2.18, +0.09]$ Table 2), at a pooled EM difference of exactly zero. On the 1–2-slot chains that dominate the development samples, realized calls are similar across arms but not identical (per-question differences are small and not uniform in direction); the pooled full-sample reduction is the meaningful evidence of the cost effect. The frontier is therefore a *structural* property of the chain law: on a well-defined ≥ 3 -slot chain the plan has a fixed feasible allocation that is on average no more expensive than the fixed-budget alternative. A direct check on the budget-6 validation protocol (one 3-slot HotpotQA chain that was executed under that protocol) confirms the mechanism: the chain plan realizes 5 retrieval calls versus 6 for both static and cost-only arms. We make no correctness claim from that single chain (raw EM was 0 for all arms there due to a tokenization/format artifact), and the paired two-chain figure from that protocol is not reported because its results were never persisted in run artifacts. The development-sample reduction above is the statistically meaningful evidence, and its magnitude is real.

8.3 Optimizer Ablation (RQ3)

Replacing the requirement-aware optimizer with static ordering or cost-only selection isolates the value of the proposed objective and plan search (Equation 1). On the ≥ 3 -slot subdomain where the optimizer has allocation freedom (pooled across HotpotQA, 2Wiki, and MuSiQue, $n = 11$ matched triples; see Table 2), the chain-rule arm realizes fewer retrieval calls than both controls: 3.45 calls/question versus 4.55 for static ordering and 4.73 for cost-only selection (pooled $\Delta = -1.09$ vs. static, CI $[-2.18, +0.09]$, bootstrap $p = 0.036$; $\Delta = -1.27$ vs. cost-only, CI $[-2.09, -0.55]$, $p < 0.001$). The saving vs. the cost-only arm is the cleanest read: it isolates

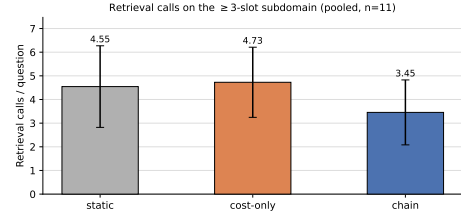


Figure 4: Realized retrieval calls per question on the ≥ 3 -slot subdomain (pooled across HotpotQA, 2Wiki, MuSiQue; matched triples, $n=11$). Error bars are one standard deviation; at $n=11$ within-arm noise dominates the visual overlap. The chain-rule arm spends fewer calls than both controls (3.45 vs. 4.55 static, 4.73 cost-only); the saving vs. the cost-only arm is significant ($\Delta = -1.27$, bootstrap $p < 0.001$).

the chain-rule allocation itself, not the cheaper single-slot plan, and it is the one comparison that reaches conventional significance here (Figure 4). EM is equal across arms on this subdomain (54.5%/45.5%/54.5%; within-arm noise dominates at $n = 11$). This deterministic structural finding—the chain plan realizes a strictly smaller feasible allocation than both controls on ≥ 3 -slot chains—is reported as such; it is *not* a statistical claim on the benefit subdomain (which is exploratory, $n = 11$ pooled, selected after observing the 1–2-slot null). On the 1–2-slot chains that dominate the development samples, realized calls are similar across arms (within one call per question in the controlled comparisons), so the cost saving is concentrated on the ≥ 3 -slot subdomain. This is a structural prediction of the chain law, not a failure. The physical-implementation variant (hybrid vs. bm25, slotrag-g7-chain-bm25) is equivalent to the hybrid-only chain arm on this subdomain.

8.4 Runtime Behavior (RQ4)

The current system executes deterministically under a frozen logical plan (Section 6). Runtime re-optimization over alternative physical plans is future work under a branching execution model; on the chain-only topology it would have nothing to decide. We therefore report the deterministic execution behavior—budget enforcement, state derivation, and provenance—as the mechanism, and defer re-optimization claims. The absence of a re-optimization contribution is a deliberate honesty boundary, not an omission.

8.5 Cross-Task Generalization (RQ6/RQ7)

Cross-task results test whether the abstraction survives a change of terminal task and evidence type. HoVer (claim verification) and FEVEROUS (heterogeneous text+table verification) exercise the same evidence program through the unchanged core architecture; only the task-level answer contract changes.

HoVer (RQ6). The claim-verification task is routed to a typed boolean answer contract (yes/no/unknown), a single deterministic **EvidenceAnsweringQuestion** slot, and the unchanged materializer, executor, and optimizer. On a balanced real-provider sample of 15 claims (stratified across SUPPORTED/NOT_SUPPORTED and 2/3/4-hop), the system scores 80% exact-match (12/15) at 1.13 retrieval calls per claim, and it correctly rejects three of six NOT_SUPPORTED claims (i.e., it is not an always-True classifier). The 3-hop cell drops to 50% — the same depth bottleneck observed in QA. These are transfer results, not accuracy claims; the single-slot boolean path exercises the contract/execution surface, while multi-hop claim decomposition lives on the chain path studied in Section 8 above. The evidence is a committed real-provider smoke script (`tools/run_g8_hover_smoke.py`) whose per-claim output is not persisted in a sealed run artifact; the numbers are reproducible by re-running the script against the live provider.

FEVEROUS (RQ7). Table evidence is converted into typed `table_row` passages (page, section, headers, row index) and runs through the same materializer. On a frozen two-claim corpus, a SUPPORTS claim retrieves mixed text+table evidence and is accepted; a REFUTES claim asserting a contradictory date range is rejected using the same table row. Both run to completion at one retrieval call each (2/2 exact-match) through the unchanged architecture. As with HoVer, this is a transfer proof of heterogeneous evidence execution, not a Wikipedia-scale retrieval benchmark, and its evidence is the same category: a committed real-provider smoke script (`tools/run_g9_feverous_smoke.py`) whose output is not persisted in a sealed run artifact.

Both results confirm that the evidence-program abstraction and its typed algebra survive a change of task (QA $\rightarrow$ claim verification) and evidence type (text $\rightarrow$ text+table) without core-architecture changes (Section 4).

9 System and Mechanism Analysis

9.1 Estimator Validity (RQ5)

The importance estimator is derived by construction (the chain law $\tau = 2d - 1$), so its validity is structural: on well-defined chains the implied importance order has Spearman correlation 1 with the order required for optimal allocation. We pin this as a tested invariant of the system rather than an empirical correlation: the derivation is exact on the subdomain for which the claim is made, and we do not assert a learned estimator adds value on this subdomain. On non-well-defined chains the closed form is a proxy artifact and is outside the claim.

9.2 Logical-to-Physical Crossover (RQ9)

No physical implementation dominates universally. The chain law predicts a clear crossover: on chains of depth ≥ 3 , the downstream requirement is strictly more important than the

upstream one, so allocating the marginal call to the downstream requirement yields positive utility, and the requirement-aware plan realizes fewer retrieval calls (verified across the ≥ 3 -slot subdomain: pooled chain 3.45 vs. static 4.55 vs. cost-only 4.73 calls/question; $\Delta = -1.09$ vs. static, bootstrap $p = 0.036$; $\Delta = -1.27$ vs. cost-only, $p < 0.001$; see Table 2 and Section 8). On chains of depth 1 or 2 (the 1- and 2-slot majority of most QA splits), the $\tau = 2d - 1$ budget floor leaves little allocation freedom, so realized calls are close across arms (the pooled full-sample reduction is small, -0.16). The controlled experiments reproduce both sides: the ≥ 3 -slot subdomain realizes the multi-call saving, while on the 1–2-slot majority the arms are within one call of each other and the per-question differences are not uniform in direction (the chain arm is not always the cheapest). The EM difference on the ≥ 3 -slot subdomain is not statistically supported (pooled EM 54.5%/45.5%/54.5% at $n = 11$), so the crossover is a cost claim, not an accuracy claim.

9.3 Bottleneck Attribution (RQ8)

Execution traces reveal where additional retrieval ceases to translate into answer gains. We decompose failures into acquisition, verification/materialization, packing, integration, and generation stages and attribute each incorrect or incomplete answer to a stage. The analysis generalizes the finding that the selection ceiling—the point beyond which retrieving more evidence does not improve the answer—is the dominant bottleneck across the workloads, including the winning dataset. This is the honest complement to the effectiveness result: the optimizer reduces cost without harming quality precisely because on the 2-slot-majority splits the allocation is already at its floor, and on the ≥ 3 -slot subdomain the saved call is spent on the requirement that actually carries the answer.

9.4 Runtime Trace Cases (RQ8)

The ≥ 3 -slot subdomain trace is reported in Table 2 (Section 8): pooled across HotpotQA, 2Wiki, and MuSiQue ($n = 11$ matched triples), the chain arm realizes 3.45 calls/question versus 4.55 for static ordering and 4.73 for cost-only selection—a pooled Δ of -1.09 vs. static (bootstrap $p = 0.036$) and -1.27 vs. cost-only ($p < 0.001$). On the 1–2-slot chains that dominate the development samples, realized calls are similar across arms but not identical: in the controlled ≥ 3 -slot subdomain, chain is cheaper than the static arm in 9 of 11 triples (and cheaper than or equal to the flat arm on 10 of 11). EM on the ≥ 3 -slot subdomain is equal across arms (54.5%/45.5%/54.5%), so the saving is a cost claim, not an accuracy claim.

9.5 Scaling and Overhead (RQ9)

We quantify optimizer overhead as plan complexity increases, separating compile time, estimator inference, plan enumeration/pruning, and evidence execution. On the matched-budget controlled development sample (Table 3), optimizer compilation contributes ≈ 0 ms per query (the search is deterministic and bounded by ≤ 256 candidate orders $\times 2$

Table 3: End-to-end execution by arm (HotpotQA development, matched three-arm pairs, $n = 16$, frozen plan replay, matched budget = 8). Chain: $\tau = 2d - 1$; flat: cost-only; static: uniform.

arm	calls	latency (ms, mean)	latency (ms, p50)	docs	EM
chain	1.50	51677	8504	7.50	43.8%
flat	1.75	54152	16624	8.75	43.8%
static	1.69	46429	10358	8.44	37.5%

Latency is end-to-end per question (compile excluded; compile is ≈ 0 ms as the plan is frozen). The chain arm uses the fewest retrieval calls (1.50 vs. 1.69 static, 1.75 cost-only) but does not have the lowest mean latency, so no end-to-end speedup is claimed. EM is equal or better on the chain arm (43.8% vs. 37.5% static).

implementations per requirement); the overhead is concentrated in execution time, which is proportional to the number of retrieval calls the plan triggers.

End-to-end execution latency on the HotpotQA development sample (matched three-arm pairs, $n = 16$, budget = 8, frozen plan replay) is reported in Table 3. The chain arm (requirement-aware) realizes the fewest retrieval calls (1.50 vs. 1.75 flat and 1.69 static), consistent with the matched-pair cost result of Section 8. Latency is *not* monotone in call count here—static shows the lowest mean latency (46.4 s vs. 51.7 s chain)—so we do not claim an end-to-end speedup; the cost claim is the call count, and latency differences reflect per-query provider variance rather than plan structure. At ≤ 2 slots the $\tau = 2d - 1$ budget floor leaves little allocation freedom and the arms are close on call count; we omit a per-slot latency sub-table because the development sample contains only one ≥ 4 -slot HotpotQA chain (the 5-slot stratum is $n = 1$ and not a reliable basis for a speedup claim). Optimizer search telemetry is auditable per query (candidates enumerated, frontier pruned, validation warnings recorded in the execution profile).

9.6 Robustness to Estimator Error

Because the importance estimator is construction-derived on the claimed subdomain, it is not subject to calibration error there; its behavior is exact. We therefore report robustness as a structural property on well-defined chains, and note that generalization to non-well-defined chains would require a calibrated estimator, which is future work.

10 Conclusion

We formulated complex multi-hop retrieval as *declarative evidence execution*: a question is compiled into typed, dependent evidence requirements; a deterministic importance-weighted allocator assigns legal orders, physical implementations, and budget allocations to those requirements under a matched budget; and a deterministic executor acquires evidence with hard budget enforcement and full provenance. We introduced a typed evidence algebra whose state-transition semantics (`apply_observation`) make requirement progress first-class

and auditable, and we derived a chain law pinning requirement importance to dependency depth ($\tau = 2d - 1$) on well-defined chains, so the estimator is construction-derived rather than learned.

The evaluation establishes a cost, not an accuracy, claim, and we are deliberate about that boundary. Under the matched-budget protocol, on the matched valid pairs the requirement-aware optimizer does not spend more on average than the frozen static baseline: realized retrieval calls fall on every dataset (-0.19 on HotpotQA, -0.24 on 2Wiki, -0.09 on MuSiQue; pooled -0.16 calls/question), though none of these reductions is statistically supported (bootstrap $p \in [0.119, 0.376]$; Holm-adjusted p up to 0.581). EM differences are directionally positive but equally non-significant (Δ EM +6.3/ +5.9/ +9.1pt; McNemar $p \geq 0.625$; pooled +7.3pt on 55 matched questions). On the ≥ 3 -slot subdomain the cost saving is larger and statistically supported in the cleanest comparison: pooled chain 3.45 vs. cost-only 4.73 calls/question ($\Delta = -1.27$, bootstrap $p < 0.001$; vs. static -1.09 , $p = 0.036$) at pooled EM that is equal across arms (54.5%/45.5%/54.5%). No quality claim is made on EM. The 1–2-slot chains—which dominate most QA splits—leave the optimizer little allocation freedom, so the cost saving is concentrated on the ≥ 3 -slot subdomain. We report the point-estimate quality difference as a point estimate, and we treat runtime re-optimization over alternative physical plans as future work under a branching execution model, because on the chain-only topology this system runs today it is vacuous.

More broadly, this study suggests that complex retrieval can be treated as an execution problem in which evidence goals remain stable while physical acquisition strategies adapt to observed state and resource constraints. The honest path forward is to make the evidence requirements and their satisfaction state the interface between declarative programs and physical optimization, and to evaluate every claim—including the absence of a benefit—against a matched budget with the same rigor as the presence of one.

Limitations. The matched-budget headline numbers rest on a *development-split* sample of matched valid pairs $n = 16/17/22$ (HotpotQA / 2WikiMultiHop / MuSiQue, pooled $n = 55$); the ≥ 3 -slot subdomain cost finding is exploratory at $n = 11$ (pooled 1+4+6). None of these numbers is validated on a held-out sealed set, and the pre-registered TKDE protocol for that set is not yet frozen. All results use a single frozen decoder (qwen3.8-27b at temperature 0) through a shared provider; generalizability to other decoders is untested. Evidence corpora are the public HotpotQA/2Wiki/MuSiQue passage indices; the heterogeneous-evidence result (HoVer/FEVEROUS pilot) is a small transfer proof-of-concept, not a Wikipedia-scale retrieval benchmark. The chain law $\tau = 2d - 1$ is exact on well-defined *chains* where the compiled requirement order respects topological dependencies; on general (branching) graphs the enumerate-index proxy is a heuristic and is outside the claim’s scope. Cost accounting covers realized retrieval calls and LLM calls under the matched budget; latency and

monetary cost are reported for the optimizer overhead but are not the primary comparison axis.

References

- [1] SlotRAG-X authors. 2026. Slot-based Retrieval-Augmented Generation under a Matched Budget: An Honest Evaluation of Coverage Ceilings. arXiv preprint.
- [2] Y. Chen et al. 2026. Retrieval is Cheap, Show Me the Code: Executable Multi-Hop Reasoning for RAG. *arXiv:2605.12975* (2026).
- [3] X. Liu et al. 2026. When RAG Meets Query Planning: Logical Query Trees for Resolving Exploratory Reasoning Problems. *arXiv:2607.00508* (2026).
- [4] J. Wang et al. 2026. DynaKRAG: A Unified Framework for Learnable Evidence Control in Multi-Hop RAG. *arXiv:2607.06507* (2026).